Ders 1 Çalışma Özeti

Ders 1 Çalışma Özeti

Genel Konular

- Dersin kapsamı ve kaynakları
  - Kapak slaytı: “Introduction, BLM5106 - Advanced Algorithm Analysis and Design” (ekrandan, 00:07:00).
  - Kaynak kitaplar (ekrandan, 00:07:58): Anany Levitin, “Introduction to the Design & Analysis of Algorithms” (2011); Cormen, Leiserson, Rivest, Stein, “Introduction to Algorithms, 3E” (MIT Press, 2009); R. Sedgewick ve K. Wayne, “Algorithms, Fourth Edition” (2013, algs4.cs.princeton.edu); Steven Skiena, “The Algorithm Design Manual” (2010). Hoca ilk kitabın çok iyi düzenlendiğini ve sık faydalanacağını söyledi.
- Dönem boyunca işlenecek konular (ekrandan, 00:08:50 ve 00:12:30)
  - Algoritmaların temelleri; algoritma verimliliği analizinin temelleri ve asimptotik analiz: Big O, Big Theta, Big Omega, büyüme mertebeleri (orders of growth).
  - Özyinelemeli olmayan (non-recursive) ve özyinelemeli (recursive) algoritmaların analizi: çalışma zamanı, recurrence relation, backward substitution ile kapalı form bulma.
  - Böl ve fethet (divide and conquer) algoritmalarının analizi; brute force, exhaustive search ve decrease and conquer ile karşılaştırma. Hoca transform and conquer’dan da söz etti.
  - Hashing algoritmaları: hoca open hashing (bağlı listelerle) ve closed hashing’e değinileceğini söyledi.
  - Dinamik programlama: rod cutting, matrix-chain multiplication, dinamik programlamanın öğeleri, optimal binary search trees.
  - Açgözlü (greedy) algoritmalar: activity-selection problem, açgözlü stratejinin öğeleri.
  - Amortize analiz: aggregate analysis, accounting method, potential method, dynamic tables.
  - İleri veri yapıları: B-trees, Fibonacci heaps, AVL trees.
  - Temel graf algoritmaları: graf gösterimleri, breadth-first search, depth-first search, topological sort, strongly connected components.
  - String matching: naive algoritma, Rabin-Karp, sonlu otomatla eşleştirme, Knuth-Morris-Pratt.
  - Yaklaşık (approximation) algoritmalar: vertex-cover, traveling-salesman, set-covering, randomization ve linear programming, subset-sum.
- Algoritma kavramı (ekrandan, 00:15:30)
  - Algoritma, bir problemi çözmek için, geçerli (legitimate) her girdiden istenen çıktıyı sonlu bir sürede elde eden, belirsizlik içermeyen komutlar dizisidir.
  - Slayttaki şema: problem → algoritma → “bilgisayar”; girdi bilgisayara girer, çıktı üretilir.
- Bir algoritmayı değerlendirme ölçütleri (ekrandan, 00:16:10)
  - Algoritma ne kadar iyi: doğruluk (correctness), verimlilik (zaman ve bellek), sadelik (simplicity), genellik (generality).
  - Slayttaki not: polinom çalışma zamanına sahip algoritma verimli sayılır.
  - Daha iyi bir algoritma var mı sorusu: alt sınırlar (lower bounds) ve optimallik.
- Temel veri yapıları (ekrandan, 00:16:40)
  - Doğrusal yapılar (dizi, bağlı liste, yığın, kuyruk), graflar (düğüm, kenar, komşuluk matrisi ve komşuluk listesi), ağaçlar (köklü ve sıralı ağaçlar), kümeler ve sözlükler (evrensel küme, bit vektörü, liste yapısı).

- Çalışma zamanını etkileyen etkenler (ekrandan, 00:17:30)
  - Sistemden bağımsız etken: algoritmanın kendisi. Bu dersin konusu budur.
  - Sisteme bağlı etkenler: donanım (CPU, bellek, önbellek), yazılım (derleyici, yorumlayıcı), sistem (işletim sistemi, ağ).
- Girdi boyutu ve çalışma zamanı birimi (ekrandan, 00:18:40)
  - Çalışma zamanını en çok etkileyen parametre girdi boyutudur (input size). Polinom değerlendirmede girdi boyutu polinomun derecesidir; slayta göre iki matrisin çarpımında çarpılan matrislerdeki toplam eleman sayısı N'dir.
  - $T(n) \approx c_{op}\, C(n)$. c_{op} temel işlemin (basic operation) bir kez çalışma süresi, $C(n)$ bu işlemin kaç kez yapıldığıdır. Sonuç yaklaşıktır.
- Büyüme mertebeleri tablosu (ekrandan, 00:21:40)
  - Sabit, logaritmik, doğrusal, linearithmic, karesel, kübik ve üstel sınıfların tipik kod yapısı, örneği ve $T(2N)/T(N)$ oranı.
- En kötü, en iyi ve ortalama durum (ekrandan, 00:23:50)
  - Sıralı arama (sequential search) üzerinden: $C_{worst}(n) = n$, $C_{best}(n) = 1$, ortalama durum olasılık varsayımlarıyla hesaplanır.

Hocanın Özellikle Vurguladığı Kısımlar

- Analizde ilgilenilen şey büyüme oranıdır
  - Sabit çarpanlar ve temel işlemin çalışma süresi (c_{op}) önemsenmez. Önemli olan, girdi boyutu büyüdüğünde işlem sayısının kaç kat arttığıdır.
  - n sonsuza giderken davranış ölçüldüğü için yalnız en yüksek dereceli terim tutulur; bu bir şey kaybettirmez.
- Hangi büyüme mertebesi tercih edilir
  - Tercih logaritmik, olmazsa doğrusaldır. Karesel kabul edilebilir, ama n arttıkça çok hızlı büyür.
  - Üstel algoritmalar ya mutlaka optimize edilmeli ya da yalnız çok küçük girdi boyutlarında kullanılmalıdır.
- En iyi durum da işe yarar
  - Genelde en kötü durum dikkate alınır. Ama neredeyse sıralı bir diziyi sıralayan insertion sort gibi durumlarda en iyi duruma yakın senaryo gerçekleşir; o zaman en iyi durumu değerlendirmek daha faydalıdır.
- Ortalama durumda asıl önemli olan p'nin seçimi
  - Formülü hesaplamaktan çok, başarılı arama olasılığı p'nin iyi belirlenmesi önemlidir.
  - p seçilirken dizinin dağılımı göz önüne alınmalıdır. Örneğin Gauss dağılımına uyan bir dizide her eleman için ayrı olasılık yerine dizi küçük parçalara bölünüp her parça için bir p belirlenebilir.

Kısa Tekrar Notları

- Algoritma: geçerli her girdi için sonlu sürede istenen çıktıyı veren, belirsizlik içermeyen komutlar dizisi.
- İyi algoritma: doğru, zaman ve bellek açısından verimli, sade, genel.
- $T(n) \approx c_{op}\, C(n)$.
- $C(n) = \frac{1}{2}n(n-1)$ ise girdi iki katına çıkınca çalışma süresi yaklaşık 4 katına çıkar.
- $T(2N)/T(N)$: sabit 1, logaritmik ~1, doğrusal 2, $N \log N$ ~2, karesel 4, kübik 8.
- Sıralı arama: en kötü n, en iyi 1, ortalama $p\frac{n+1}{2} + n(1-p)$.
- $p = 1$ ise ortalama $\frac{n+1}{2}$; $p = 0$ ise ortalama n.

Detaylı Açıklamalar

- Girdi boyutu iki katına çıkınca ne olur (ekrandan, 00:18:40)
  - Soru: $C(n) = \frac{1}{2}n(n-1)$ ise girdi boyutu iki katına çıktığında algoritma ne kadar uzun çalışır?
  - Önce açılır: $C(n) = \frac{1}{2}n^2 - \frac{1}{2}n \approx \frac{1}{2}n^2$. İki terimden üssü yüksek olan baz alınır.

  - Oran: $\frac{T(2n)}{T(n)} \approx \frac{c_{op}C(2n)}{c_{op}C(n)} \approx \frac{\frac{1}{2}(2n)^2}{\frac{1}{2}n^2} = 4$. c_{op}, $\frac{1}{2}$ ve n^2 sadeleşir.
  - Bu sonuç, analizde sabitlerin ve temel işlem süresinin değil, büyüme oranının önemli olduğunu gösterir. Bu orana büyüme mertebesi (order of growth) denir.
- Büyüme mertebeleri tablosu (ekrandan, 00:21:40)
  - Sabit (1): `a = b + c;` gibi tek bir ifade, örnek iki sayının toplanması, oran 1.
  - Logaritmik ($\log N$): `while (N > 1) { N = N / 2; ... }`, her adımda yarıya bölme, örnek binary search, oran ~1.
  - Doğrusal (N): tek `for` döngüsü, örnek en büyük elemanı bulma (hoca sıralı aramayı söyledi), oran 2.
  - Linearithmic ($N \log N$): böl ve fethet, örnek mergesort, oran ~2.
  - Karesel (N^2): iç içe iki döngü, tüm çiftleri kontrol etme (hoca: dizide aynı olan çiftleri bulma), oran 4.
  - Kübik (N^3): iç içe üç döngü, tüm üçlüleri kontrol etme, oran 8.
  - Üstel (2^N): exhaustive search, tüm alt kümeleri kontrol etme. Tabloda oran sütunu boştur.
- Sıralı arama algoritması (ekrandan, 00:23:50)

```
ALGORITHM SequentialSearch(A[0..n-1], K)
// Girdi: A[0..n-1] dizisi ve arama anahtarı K
// Çıktı: A'da K ile eşleşen ilk elemanın indisi, yoksa -1
i <- 0
while i < n and A[i] != K do
    i <- i + 1
if i < n return i
else return -1
```

  - En kötü durum $C_{worst}(n) = n$: anahtar dizide yoksa ya da son elemansa bütün dizi taranır.
  - En iyi durum $C_{best}(n) = 1$: anahtar ilk elemandır.
- Ortalama durumun hesabı (ekrandan, 00:26:40)
  - Varsayımlar: (a) başarılı arama olasılığı p, $0 \leq p \leq 1$; (b) ilk eşleşmenin i. konumda olma olasılığı her i için aynıdır.
  - Bu durumda her konumda bulunma olasılığı $\frac{p}{n}$'dir. i. konumda bulmak için döngü i kez döner.
  - Bulunamama olasılığı $1 - p$'dir ve bu durumda n karşılaştırma yapılır.
  - $C_{avg}(n) = \left[1 \cdot \frac{p}{n} + 2 \cdot \frac{p}{n} + \cdots + i \cdot \frac{p}{n} + \cdots + n \cdot \frac{p}{n}\right] + n(1-p)$.
  - Sadeleştirme (ara adım eklendi): $\frac{p}{n} \cdot \frac{n(n+1)}{2} + n(1-p) = \frac{p(n+1)}{2} + n(1-p)$.
  - $p = 1$ (anahtar kesin dizide): $\frac{n+1}{2}$; ortalama olarak dizinin ortasına gelince bulunur.
  - $p = 0$ (anahtar dizide yok): köşeli parantezli kısım sıfır olur, sonuç n'dir.
  - Ortalama durum, en kötü durumun fazla karamsar olduğu düşüncesiyle daha gerçekçi bir sonuç arar; hesabı en kötü ve en iyi duruma göre daha karmaşıktır.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.